Assembly Language for x86 Processors 7th Edition , Global Edition

Kip R. Irvine

Chapter 16: BIOS-Level Programming

(c) Pearson Education, 2015. All rights reserved. You may modify and copy this slide show for your personal use, or for use in the classroom, as long as this copyright statement, the author's name, and the title are not changed.

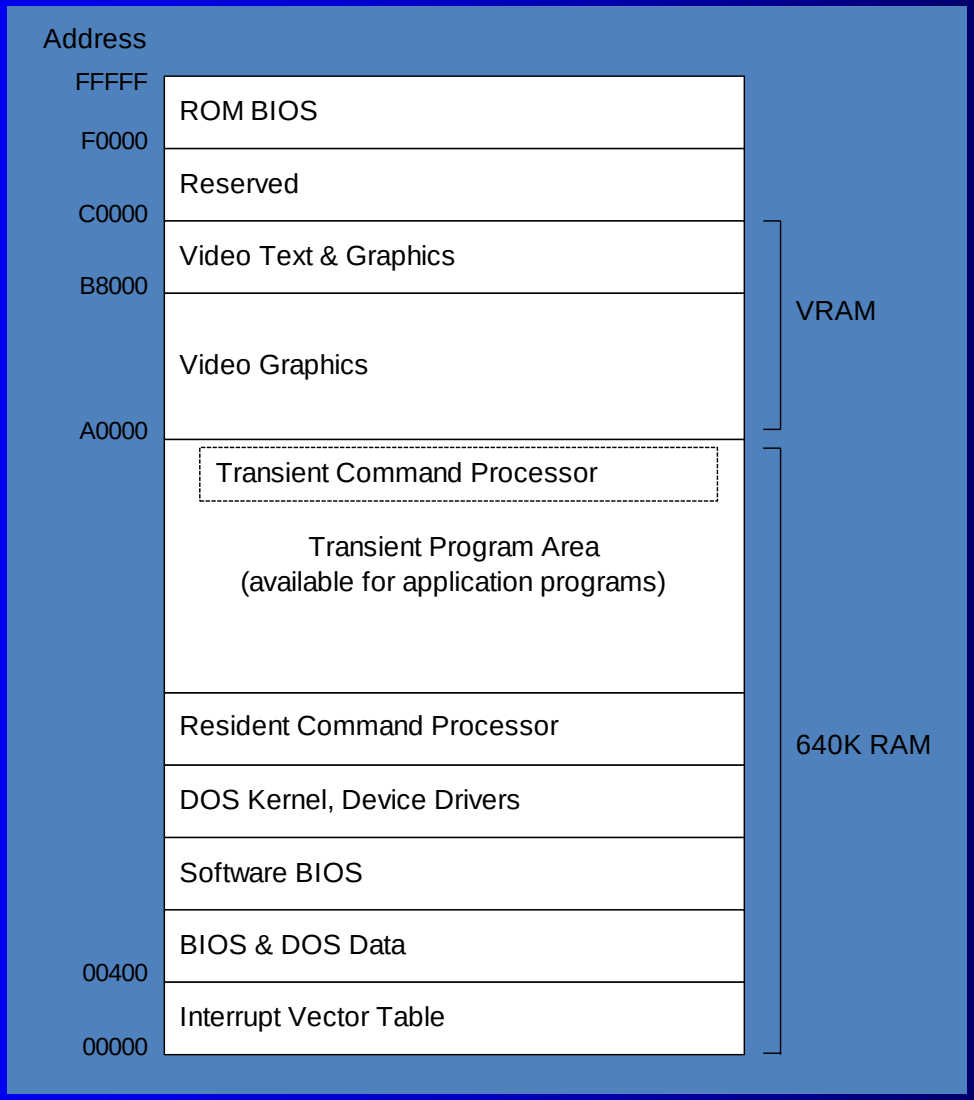
Chapter Overview

- Introduction
- Keyboard Input with INT 16h
- VIDEO Programming with INT 10h
- Drawing Graphics Using INT 10h
- Memory-Mapped Graphics
- Mouse Programming

PC-BIOS

- The BIOS (Basic Input-Output System) provides low-level hardware drivers for the operating system.
  - accessible to 16-bit applications
  - written in assembly language, of course
  - source code published by IBM in early 1980's
- Advantages over MS-DOS:
  - permits graphics and color programming
  - faster I/O speeds
  - read mouse, serial port, parallel port
  - low-level disk access

(Recap) MS-DOS Memory Map



BIOS Data Area

- Fixed-location data area at address 00400h
  - this area is also used by MS-DOS
  - this area is accessible under Windows 98 & Windows Me, but not under Windows NT, 2000, or XP.
- Contents:
  - Serial and parallel port addresses
  - Hardware list, memory size
  - Keyboard status flags, keyboard buffer pointers, keyboard buffer data
  - Video hardware configuration
  - Timer data

BIOS Data Area, at Segment 0040h

Hex Offset	Description
0000 – 0007	Port addresses, COM1 – COM4
0008 – 000F	Port addresses, LPT1 – LPT4
0010 – 0011	Installed hardware list
0012	Initialization flag
0013 – 0014	Memory size, in kilobytes
0015 – 0016	Memory in I/O channel
0017 – 0018	Keyboard status flags
0019	Alternate key entry storage
001A – 001B	Keyboard buffer pointer (head)
001C – 001D	Keyboard buffer pointer (tail)
001E – 003D	Keyboard typeahead buffer
003E – 0048	Diskette data area
0049	Current video mode
004A – 004B	Number of screen columns
004C – 004D	Regen (video) buffer length, in bytes
004E – 004F	Regen (video) buffer starting offset
0050 – 005F	Cursor positions, video pages 1 – 8
0060	Cursor end line
0061	Cursor start line
0062	Currently displayed video page number
0063 – 0064	Active display base address
0065	CRT mode register
0066	Register for color graphics adapter
0067 – 006B	Cassette data area
006C – 0070	Timer data area

What's Next

- Introduction
- **Keyboard Input with INT 16h**
- VIDEO Programming with INT 10h
- Drawing Graphics Using INT 10h
- Memory-Mapped Graphics
- Mouse Programming

Keyboard Input with INT 16h

- How the Keyboard Works
- INT 16h Functions

KEYBOARD SCAN CODES(1/2)

KEYBOARD SCAN CODES

The following keyboard scan codes may be retrieved either by calling INT 16h or by calling INT 21h for keyboard input a second time (the first keyboard read returns 0). All codes are in hexadecimal:

FUNCTION KEYS

Key	Normal	With Shift	With Ctrl	With Alt
F1	3B	54	5E	68
F2	3C	55	5F	69
F3	3D	56	60	6A
F4	3E	57	61	6B
F5	3F	58	62	6C
F6	40	59	63	6D
F7	41	5A	64	6E
F8	42	5B	65	6F
F9	43	5C	66	70
F10	44	5D	67	71
F11	85	87	89	8B
F12	86	88	8A	8C

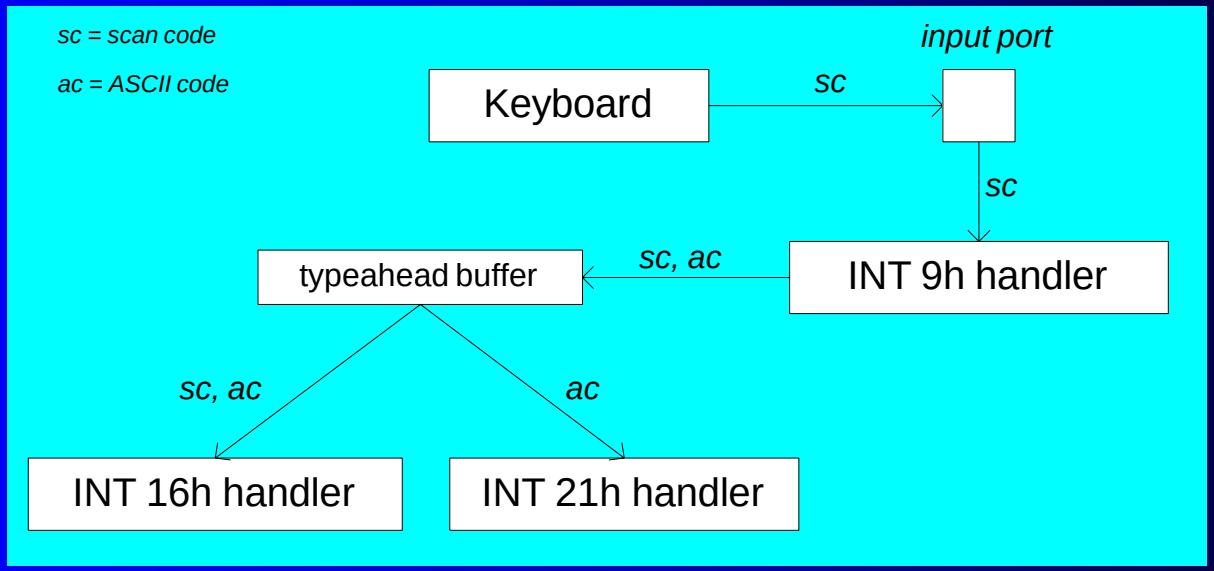
Irvine, Kip R. Assembly Language for Intel-Based Computers 5/e, 2007.

KEYBOARD SCAN CODES(2/2)

Key	Alone	With Ctrl Key
Home	47	77
End	4F	75
PgUp	49	84
PgDn	51	76
PrtSc	37	72
Left arrow	4B	73
Rt arrow	4D	74
Up arrow	48	8D
Dn arrow	50	91
Ins	52	92
Del	53	93
Back tab	0F	94
Gray +	4E	90
Gray –	4A	8E

How the Keyboard Works

- Keystroke sends a scan code to the keyboard serial input port
- Interrupt triggered: INT 9h service routine executes
- Scan code and ASCII code inserted into keyboard typeahead buffer



PC Interrupts

D.2 PC Interrupts

Table D-1 General List of PC Interrupt Numbers.[a]

Number	Description
0	*Divide Error*. CPU-generated: activated when attempting to divide by zero.
1	*Single Step*. CPU-generated: active when the CPU Trap flag is set.
2	*Nonmaskable Interrupt*. External hardware: activated when a memory error occurs.
3	*Breakpoint*. CPU-generated: activated when the 0CCh (INT 3) instruction is executed.
4	*INTO Detected Overflow*. CPU-generated: Activated when the INTO instruction is executed and the Overflow flag is set.
5	*Print Screen*. Activated either by the INT 5 instruction or pressing the Shift-PrtSc keys.
6	*Invalid OpCode* (80286+)
7	*Processor Extension Not Available* (80286+)
8	IRQ0: *System Timer Interrupt*. Updates the BIOS clock 18.2 times per second. For your own programming, see INT 1Ch.
9	IRQ1: *Keyboard Hardware Interrupt*. Activated when a key is pressed. Reads the key from the keyboard port and stores it in the keyboard typeahead buffer.
0A	IRQ2: *Programmable Interrupt Controller*
0B	IRQ3: Serial Communications (COM2)
0C	IRQ4: Serial Communications (COM1)
0D	IRQ5: Fixed Disk
0E	IRQ6: *Diskette Interrupt*. Activated when a disk seek is in progress.
0F	IRQ7: *Parallel Printer*

PC Interrupts (cont.)

Table D-1 *(Continued)*

Number	Description
19	*Bootstrap Loader.* Reboot MS-DOS.
1A	*Time of Day.* Get the number of timer ticks since the machine was turned on, or set the counter to a new value. Ticks occur 18.2 times per second.
1B	*Keyboard Break*. This interrupt handler is executed by INT 9h when CTRL-BREAK is pressed.
1C	*User Timer Interrupt*. Empty routine, executed 18.2 times per second. May be used by your own program.
1D	*Video Parameters*. Point to a table containing initialization and information for the video controller chip.
1E	*Diskette Parameters*. Point to a table containing initialization information for the diskette controller.
1F	*Graphics Table*. 8 × 8 Graphics font. Table kept in memory of all extended graphics characters with ASCII codes higher than 127.
20	*Terminate Program*. Terminate a COM program (INT 21h Function 4Ch should be used instead).
21	*MS-DOS Services* (see the complete list in Table D-2).
22	*MS-DOS Terminate Address*. Point to the address of the parent program or process. When the current program ends, this will be the return address.

PC Interrupts (cont.)

23	*MS-DOS Break Address*. MS-DOS jumps here when CTRL-BREAK is pressed.
24	*MS-DOS Critical Error Address*. DOS jumps to this address when there is a critical error in the current program, such as a disk media error.
25	*Absolute Disk Read* (obsolete).
26	*Absolute Disk Write* (obsolete).
27	*Terminate and Stay Resident* (obsolete).
28–FF	(Reserved)
33	*Microsoft Mouse*. Functions that track and control the mouse.
34–3E	*Floating-Point Emulation*.
3F	*Overlay manager*.
40–41	*Fixed Disk Services*. Fixed disk controller.
42–5F	Reserved: specialized uses
60–6B	Available for application programs to use.
6C–7F	Reserved: specialized uses
80–F0	Reserved: used by ROM BASIC.
F1–FF	Available for application programs.

[a]Sources: Ray Duncan, *Advanced MS-DOS*, 2nd ed., Microsoft Press, 1998. *Ralf Brown's Interrupt List*, available on the Web.

INT 21h Appendix

D.3 Interrupt 21H Functions (MS-DOS Services)

There are so many MS-DOS services available through INT 21h that we could not possibly document them all here. Instead, Table D-2 is simply a brief overview of functions that are commonly used.

Table D-2 Interrupt 21h Functions (MS-DOS Services).

Function	Description
1	*Read character from standard input*. If no character is ready, wait for input. Returns: AL = character.
2	*Write character to standard output*. Receives: DL = character.
3	*Read character from standard auxiliary input* (serial port).
4	*Write character to standard auxiliary output* (serial port).
5	*Write character to printer*. Receives: DL = character.
6	*Direct console input/output*. If DL = FFh, read a waiting character from standard input. If DL is any other value, write the character in DL to standard output.
7	*Direct character input without echo*. Wait for a character from standard input. Returns: AL = character.
8	*Character input without echo*. Wait for a character from the standard input device. Returns: AL = character. Character not echoed. May be terminated by Ctrl-Break.
9	*Write string to standard output*. Receives: DS:DX = address of string.
0A	*Buffered keyboard input*. Read a string of characters from the standard input device. Receives: DS:DX points to a predefined keyboard structure.

INT 21h Appendix (cont.)

0B	*Check standard input status*. Check to see if an input character is waiting. Returns: AL = 0FFh if the character is ready; otherwise, AL = 0.
0C	*Clear keyboard buffer and invoke input function*. Clear the console input buffer, and then execute an input function. Receives: AL = desired function (1, 6, 7, 8, or 0Ah).
0E	*Select default drive*. Receives: DL = drive number (0 = A, 1 = B, etc.).
0F–18	FCB file functions (obsolete).
19	*Get current default drive*. Returns: AL = drive number (0 = A, 1 = B, etc.)
1A	*Set disk transfer address*. Receives: DS:DX contains address of disk transfer area.
25	*Set interrupt vector*. Set an entry in the Interrupt Vector Table to a new address. Receives: DS:DX points to the interrupt-handling routine that is inserted in the table; AL = the interrupt number.
26	*Create new program segment prefix*. Receives: DX = segment address for new PSP.
27–29	FCB file functions (obsolete).
2A	*Get system date*. Returns: AL = Day of the week (0–6, where Sunday = 0), CX = year, DH = month, and DL = day.
2B	*Set system date*. Receives: CX = year, DH = month, and DL = day. Returns: AL = 0 if the date is valid.
2C	*Get system time*. Returns: CH = hour, CL = minutes, DH = seconds, and DL = hundredths of seconds.
2D	*Set system time*. Receives: CH = hour, CL = minutes, DH = seconds, and DL = hundredths of seconds. Returns: AL = 0 if the time is valid.
2E	*Set Verify flag*. Receives: AL = new state of MS-DOS Verify flag (0 = off, 1 = on), DL = 00h.

INT 21h Appendix (cont.)

Table D-2 *(Continued)*

Function	Description
2F	*Get disk transfer address* (DTA). Returns: ES:BX = address.
30	*Get MS-DOS version number*. Returns: AL = major version number, AH = minor version number, BH = OEM serial number, BL:CX = 24-bit user serial number.
31	*Terminate and stay resident*. Terminate the current program or process, leaving part of itself in memory. Receives: AL = return code, and DX = requested number of paragraphs.
32	*Get MS-DOS drive parameter block*. Receives: DL = drive number. Returns: AL = status; DS:BX points to drive parameter block.
33	*Extended break checking*. Indicates whether or not MS-DOS is checking for Ctrl-Break.
34	*Get address of INDOS flag*. (Undocumented)
35	*Get interrupt vector*. Receives: AL = interrupt number. Returns: ES:BX = segment/offset of the interrupt handler.
36	*Get disk free space*. (FAT16 only) Receives: DL = drive number (0 = default, 1 = A, etc.). Returns: AX = sectors per cluster, or FFFFh if the drive number is invalid; BX = number of available clusters, CX = bytes per sector, and DX = clusters per drive.
37	*Get switch character*. (Undocumented)
38	*Get or set country information*.[a]
39	*Create subdirectory*. Receives: DS:DX points to an ASCIIZ string with the path and directory name. Returns: AX = error code if the Carry flag is set.

INT 21h Appendix (cont.)

3A	*Remove subdirectory.* Receives: DS:DX points to an ASCIIZ string with the path and directory name. Returns: AX = error code if the Carry flag is set.
3B	*Change current directory.* Receives: DS:DX points to an ASCIIZ string with the new directory path. Returns: AX = error code if the Carry flag is set.
3C	*Create or truncate file.* Create a new file or truncate an old file to zero bytes. Open the file for output. Receives: DS:DX points to an ASCIIZ string with the file name, and CX = file attribute. Returns: AX = error code if the Carry flag is set; otherwise AX = the new file handle.
3D	*Open existing file*. Open a file for input, output, or input-output. Receives: DS:DX points to an ASCIIZ string with the filename, and AL = the access code (0 = read, 1 = write, 2 = read/write). Returns: AX = error code if the Carry flag is set, otherwise AX = the new file handle.
3E	*Close file handle.* Close the file or device specified by a file handle. Receives: BX = file handle from previous open or create. Returns: If the Carry Flag is set, AX = error code.
3F	*Read from file or device.* Read a specified number of bytes from a file or device. Receives: BX = file handle, DS:DX points to an input buffer, and CX = number of bytes to read. Returns: If the Carry flag is set, AX = error code; otherwise, AX = number of bytes read.
40	*Write to file or device*. Write a specified number of bytes to a file or device. Receives: BX = file handle, DS:DX points to an output buffer, and CX = the number of bytes to write. Returns: If the Carry flag is set, AX = error code; otherwise, AX = number of bytes written.
41	*Delete file*. Remove a file from a specified directory. Receives: DS:DX points to an ASCIIZ string with the filename. Returns: AX = error code if the Carry flag is set.

INT 21h Appendix (cont.)

Table D-2 *(Continued)*

Function	Description
42	*Move file pointer*. Move the file read/write pointer according to a specified method. Receives: CX:DX = distance (bytes) to move the file pointer, AL = method code, BX = file handle. The method codes are as follows: 0 = move from beginning of file, 1 = move to the current location plus an offset, and 2 = move to the end of file plus an offset. Returns: AX = error code if the Carry flag is set.
43	*Get/Set file attribute*. Get or set the attribute of a file. Receives: DS:DX = pointer to an ASCIIZ path and filename, CX = attribute, and AL = function code (1 = set attribute, 0 = get attribute). Returns: AX = error code if the Carry flag is set.
44	*I/O control for devices*. Get or set device information associated with an open device handle, or send a control string to the device handle, or receive a control string from the device handle.
45	*Duplicate file handle*. Return a new file handle for a file that is currently open. Receives: BX = file handle. Returns: AX = error code if the Carry flag is set.
46	*Force duplicate file handle*. Force the handle in CX to refer to the same file at the same position as the handle in BX. Receives: BX = existing file handle and CX = second file handle. Returns: AX = error code if the Carry flag is set.
47	*Get current directory*. Get the full path name of the current directory. Receives: DS:SI points to a 64-byte area to hold the directory path, and DL = drive number. Returns: A buffer at DS:SI is filled with the path, and AX = error code if the Carry flag is set.
48	*Allocate memory*. Allocate a requested number of paragraphs of memory, measured in 16-byte blocks. Receives: BX = number of paragraphs requested. Returns: AX = segment of the allocated block and BX = size of the largest block available (in paragraphs), and AX = error code if the Carry flag is set.

INT 21h Appendix (cont.)

49	*Free allocated memory.* Free memory that was previously allocated by Function 48h. Receives: ES = segment of the block to be freed. Returns: AX = error code if the Carry flag is set.
4A	*Modify memory blocks.* Modify allocated memory blocks to contain a new block size. The block will shrink or grow. Receives: ES = segment of the block and BX = requested number of paragraphs. Returns: AX = error code if the Carry flag is set and BX = maximum number of available blocks.
4B	*Load or execute program.* Create a program segment prefix for another program, load it into memory, and execute it. Receives: DS:DX points to an ASCIIZ string with the drive, path, and filename of the program; ES:BX points to a parameter block and AL = function value. Function values in AL:0 = load and execute the program; 3 = load but do not execute (overlay program). Returns: AX = error code if the Carry flag is set.
4C	*Terminate process.* Usual way to terminate a program and return to either MS-DOS or a calling program. Receives: AL = 8-bit return code, which can be queried by DOS function 4Dh or by the ERRORLEVEL command in a batch file.
4D	*Get return code of process.* Get the return code of a process or program, generated by either function call 31h or function call 4Ch. Returns: AL = 8-bit code returned by the program, AH = type of exit generated: 0 = normal termination, 1 = terminated by CTRLBREAK, 2 = terminated by a critical device error, and 3 = terminated by a call to function call 31h.
4E	*Find first matching file.* Find the first filename that matches a given file specification. Receives: DS:DX points to an ASCIIZ drive, path, and file specification; CX = File attribute to be used when searching. Returns: AX = error code if the Carry flag is set; otherwise, the current DTA is filled with the filename, attribute, time, date, and size. DOS function call 1Ah (set DTA) is usually called before this function.

Keyboard Flags

16-bits, located at 0040:0017h – 0018h.

Bit	Description
0	Right Shift key is down
1	Left Shift key is down
2	Either Ctrl key is down
3	Either Alt key is down
4	Scroll Lock toggle is on
5	Num Lock toggle is on
6	Caps Lock toggle is on
7	Insert toggle is on
8	Left Ctrl key is down

Bit	Description
9	Left Alt key is down
10	Right Ctrl key is down
11	Right Alt key is down
12	Scroll key is down
13	Num Lock key is down
14	Caps Lock key is down
15	SysReq key is down

INT 16h Functions

- Provide low-level access to the keyboard, more so than MS-DOS.
- Function number is always in the AH register
- Important functions:
  - set typematic rate
  - push key into buffer
  - wait for key
  - check keyboard buffer
  - get keyboard flags

Function 10h: Wait for Key

If a key is waiting in the buffer, the function returns it immediately. If no key is waiting, the program pauses (blocks), waiting for user input.

```
.data
scanCode  BYTE ?
ASCIICode BYTE ?

.code
mov ah,10h
int 16h
mov scanCode,ah
mov ASCIICode,al
```

Function 12h: Get Keyboard Flags

Retrieves a copy of the keyboard status flags from the BIOS data area.

```
.data
keyFlags WORD ?

.code
mov ah,12h
int 16h
mov keyFlags,ax
```

Clearing the Keyboard Buffer

Function 11h clears the Zero flag if a key is waiting in the keyboard typeahead buffer.

```
L1: mov ah,11h                  ; check keyboard buffer
    int 16h                     ; any key pressed?
    jz  noKey                   ; no: exit now
    mov ah,10h                  ; yes: remove from buffer
    int 16h
    cmp ah,scanCode             ; was it the exit key?
    je  quit                    ; yes: exit now (ZF=1)
    jmp L1                      ; no: check buffer again

noKey:                          ; no key pressed
    or  al,1                    ; clear zero flag
quit:
```

Clearing the Keyboard Buffer

ASCII Code*	Ctrl-	Mnemonic	Description	ASCII Code*	Ctrl-	Mnemonic	Description
00		NUL	Null character	10	Ctrl-P	DLE	Data link escape
01	Ctrl-A	SOH	Start of header	11	Ctrl-Q	DC1	Device control 1
02	Ctrl-B	STX	Start of text	12	Ctrl-R	DC2	Device control 2
03	Ctrl-C	ETX	End of text	13	Ctrl-S	DC3	Device control 3
04	Ctrl-D	EOT	End of transmission	14	Ctrl-T	DC4	Device control 4
05	Ctrl-E	ENQ	Enquiry	15	Ctrl-U	NAK	Negative acknowledge
06	Ctrl-F	ACK	Acknowledge	16	Ctrl-V	SYN	Synchronous idle
07	Ctrl-G	BEL	Bell	17	Ctrl-W	ETB	End transmission block
08	Ctrl-H	BS	Backspace	18	Ctrl-X	CAN	Cancel
09	Ctrl-I	HT	Horizontal tab	19	Ctrl-Y	EM	End of medium
0A	Ctrl-J	LF	Line feed	1A	Ctrl-Z	SUB	Substitute
0B	Ctrl-K	VT	Vertical tab	1B	Ctrl-I	ESC	Escape
0C	Ctrl-L	FF	Form feed	1C	Ctrl-\	FS	File separator
0D	Ctrl-M	CR	Carriage return	1D	Ctrl-]	GS	Group separator
0E	Ctrl-N	SO	Shift out	1E	Ctrl- ^	RS	Record separator
0F	Ctrl-O	SI	Shift in	1F	Ctrl-†	US	Unit separator

What's Next

- Introduction
- Keyboard Input with INT 16h
- **VIDEO Programming with INT 10h**
- Drawing Graphics Using INT 10h
- Memory-Mapped Graphics
- Mouse Programming

Three Levels of Video Access

- MS-DOS function calls
  - slow, but they work on any MS-DOS machine
- BIOS function calls
  - medium-fast, work on nearly all MS-DOS-based machines
- Direct memory-mapped video
  - fast – works only on 100% IBM-compatible computers

Controlling the Color

- Mix primary colors: red, green, blue
  - called subtractive mixing
  - add the intensity bit for 4th channel
- Examples:
  - red + green + blue = light gray (0111)
  - intensity + red + green + blue = white (1111)
  - green + blue = cyan (0011)
  - red + blue = magenta (0101)
- Attribute byte:
  - 4 MSB bits = background
  - 4 LSB bits = foreground

Constructing Attribute Bytes

- Examples:
  - Light gray text on a blue background:
    - (blue SHL 4) OR lightGray
  - White text on a red background:
    - (red SHL 4) OR white

INT 10h Video Functions

- AH register contains the function number
- 00h: Set video mode
  - text modes
  - graphics modes
- 01h: Set cursor lines
- 02h: Set cursor position
- 03h: Get cursor position and size
- 06h: Scroll window up
- 07h: Scroll window down
- 08h: Read character and attribute

INT 10h Video Functions *(cont)*

- 09h: Write character and attribute
- 0Ah: Write character
- 10h (AL = 03h): Toggle blinking/intensity bit
- 0Fh: Get video mode

Displaying a Color String

Write one character and attribute:

```
mov   si,OFFSET string
. . .
mov   ah,9                    ; write character/attribute
mov   al,[si]                 ; character to display
mov   bh,0                    ; video page 0
mov   bl,color                ; attribute
or    bl,10000000b            ; set blink/intensity bit
mov   cx,1                    ; display it one time
int   10h
```

Clrscr Procedure

```
Clrscr PROC
   pusha
   mov     ax,0600h          ; scroll window up
   mov     cx,0              ; upper left corner (0,0)
   mov     dx,184Fh          ; lower right corner (24,79)
   mov     bh,7              ; normal attribute
   int     10h               ; call BIOS
   mov     ah,2              ; locate cursor at 0,0
   mov     bh,0              ; video page 0
   mov     dx,0              ; row 0, column 0
   int     10h
   popa
   ret
Clrscr ENDP
```

What's Next

- Introduction
- Keyboard Input with INT 16h
- VIDEO Programming with INT 10h
- Drawing Graphics Using INT 10h
- Memory-Mapped Graphics
- Mouse Programming